AWS VPC PEERING

Student Project Report

Student Name	Pratik Pradip Shinde
College	Bharati Vidyapeeth College of Engineering, Pune
Batch	[TO BE ADDED]
Project	AWS VPC Peering
AWS Region	us-east-1 (reported; verify in AWS Console)
Completion Date	31 October 2026 (working date; confirm actual submission date)

DRAFT — Evidence and a few submission details are still outstanding. Replace the marked items and insert the actual AWS screenshots before submitting.

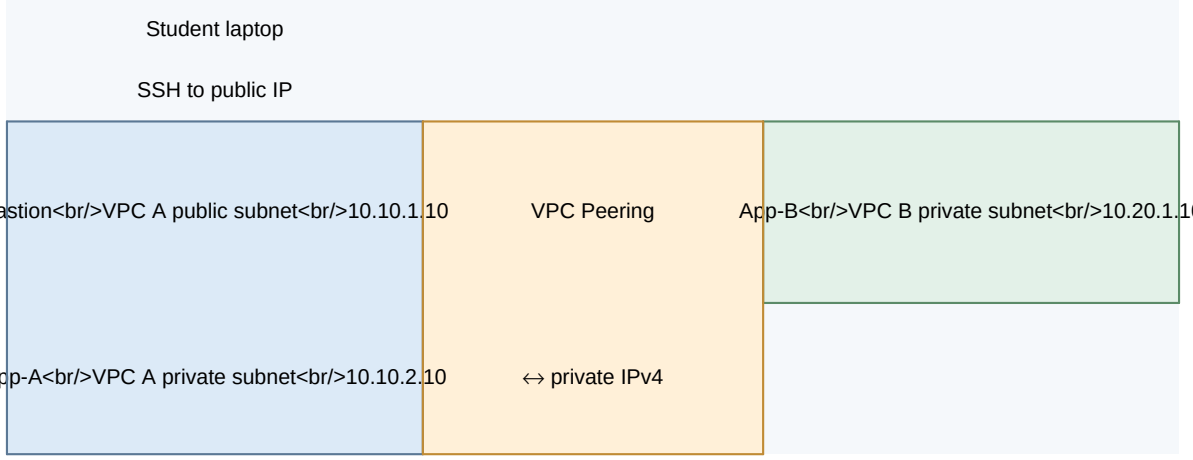
1. Project Overview

The objective of this project is to establish private network communication between two Amazon Virtual Private Clouds (VPCs) using VPC Peering. The design uses a public bastion host in VPC A to provide controlled SSH access to private EC2 instances. App-A and App-B communicate across the peering connection using private IPv4 addresses.

VPC peering provides a direct private network route between the two VPCs. Both sides need appropriate routes, and instance security groups must allow the required traffic. VPC peering is non-transitive; VPC B cannot use VPC A's Internet Gateway as an internet path. [1][2]

2. Architecture and Resource Plan

The intended architecture is: Student laptop → Bastion (public subnet in VPC A) → private workloads. App-A resides in VPC A's private subnet; App-B resides in VPC B's private subnet. VPC A and VPC B are connected by one VPC peering connection.



Resource inventory (planned values from the project workbook; confirm against the live AWS console):

Resource	Name	CIDR / Private IP
VPC A	ewb-vpc-a	10.10.0.0/16
VPC A public subnet	ewb-a-public	10.10.1.0/24
VPC A private subnet	ewb-a-private	10.10.2.0/24
VPC B	ewb-vpc-b	10.20.0.0/16
VPC B private subnet	ewb-b-private	10.20.1.0/24
Bastion EC2	ewb-bastion	10.10.1.10; public IP recorded: 32.201.8.68
App-A EC2	ewb-app-a	10.10.2.10
App-B EC2	ewb-app-b	10.20.1.10
Peering connection	ewb-a-b-peer	pcx-04eeb4357690b525a (reported)
Internet Gateway	ewb-igw-a	igw-0332b04b64cbe0349 (reported)

3. Routing Configuration

The following route design is required for private communication. The route table names and destination/target values below follow the workbook. Confirm each table's actual subnet association and route status in the AWS console.

Route table	Destination	Target
ewb-rt-a-public	10.10.0.0/16	local
ewb-rt-a-public	0.0.0.0/0	Internet Gateway (igw-0332b04b64cbe0349)
ewb-rt-a-public	10.20.0.0/16	VPC Peering (pcx-04eeb4357690b525a)
ewb-rt-a-private	10.10.0.0/16	local
ewb-rt-a-private	10.20.0.0/16	VPC Peering (pcx-04eeb4357690b525a)
ewb-rt-b-private	10.20.0.0/16	local
ewb-rt-b-private	10.10.0.0/16	VPC Peering (pcx-04eeb4357690b525a)

A peering connection being Active is not sufficient by itself: each side needs a route to the peer VPC CIDR, and the return route must also exist. [1]

4. Security Groups and Access

Expected inbound rules from the workbook (verify these against the actual security groups before final submission):

Security group	Inbound rule	Source
ewb-sg-bastion	SSH TCP 22	Your current public IPv4 /32 (captured value: 32.201.8.68/32 — confirm still curr
ewb-sg-app-a	SSH TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a	ICMP Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b	SSH TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b	ICMP Echo request	10.10.2.10/32 (App-A)

The private instances were accessed through the bastion using SSH ProxyCommand/ProxyJump from the Windows laptop. The private workloads did not need public IPv4 addresses. Do not include the PEM private key, passwords, access keys, or tokens in this report or GitHub repository.

5. Connectivity Test Results

The screenshots supplied in the conversation show SSH access to the App-B instance through the bastion and successful ICMP tests in both directions.

App-A → App-B: 4 packets transmitted, 4 received, 0% packet loss (10.10.2.10 to 10.20.1.10).

App-B → App-A: 4 packets transmitted, 4 received, 0% packet loss (10.20.1.10 to 10.10.2.10).

SSH: The terminal evidence shows a successful login to App-B through the bastion, followed by hostname and private IP verification.

Controlled failure test — NOT YET CAPTURED

The required negative test has not yet been documented. Before final submission, temporarily remove only the App-B inbound ICMP Echo request rule whose source is 10.10.2.10/32. From App-A, run `ping -c 4 10.20.1.10` and capture the failure. Then immediately restore the rule, rerun the ping, and capture the successful recovery. Do not leave the rule removed.

6. Screenshots to Insert

Filename	Required evidence
01-vpc-a.png / 02-vpc-b.png	VPC names, CIDRs and IDs
03-subnets.png	All three subnet names, CIDRs and VPC associations
04-route-tables.png	Routes and subnet associations for all three tables
05-peering-active.png	Peering connection ID and Active status
06-ssh-access.png	Bastion and private instance SSH access
07-forward-ping.png	App-A to App-B ping success
08-reverse-ping.png	App-B to App-A ping success
09-controlled-failure.png	Temporary ICMP removal and failed ping
10-recovery-success.png	Restored ICMP rule and successful ping
11-security-groups.png	Inbound rules for all three security groups
12-private-ip-proof.png	Private IP/hostname proof; no public IP on App-A/App-B
13-cleanup.png	AWS console evidence after cleanup

7. Learning Outcomes

This project demonstrates the use of VPCs and subnet CIDR planning, public and private subnet separation, Internet Gateway routing for the bastion subnet, VPC peering, bidirectional route configuration, security-group restrictions, SSH tunnelling through a bastion host, and verification of private connectivity using ICMP.

8. Cleanup and Cost Control

Cleanup is still pending confirmation. After all screenshots and the report are saved, terminate the three EC2 instances; verify that attached EBS volumes are deleted; remove any unused snapshots and Elastic IP allocations; delete the peering connection and peer routes; delete security groups after dependent network interfaces are gone; remove subnet associations; delete subnets and custom route tables; detach and delete the Internet Gateway; and finally delete both VPCs. Verify that no project resources remain. AWS billing/usage reporting can take time to update.

9. Final Submission Checklist

■ Add the internship batch on the cover page.

■ Verify AWS Region and all resource IDs directly in the AWS console.

■ Insert the actual screenshots listed in Section 6.

■ Capture the controlled failure and recovery evidence.

■ Add cleanup evidence after deleting resources.

■ Export the final PDF as Pratik_Pradip_Shinde_EWB_VPC_Peering.pdf.

■ Upload the PDF and supporting screenshots/architecture to the GitHub repository.

■ Submit the repository URL through the internship Google Form.

References

[1] AWS Documentation — VPC peering connections: https://docs.aws.amazon.com/vpc/latest/peering/working-with-vpc-peering.html

[2] AWS Documentation — How VPC peering connections work and limitations: https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-basics.html

Report status: DRAFT. Values shown as 'reported' or 'planned' must be checked against the AWS console. The report is not ready for submission until the missing evidence and batch information are added.